

Análise computacional de três problemas de escalonamento (Machine Scheduling, Job Shop e Flow Shop) em Julia + JuMP com o solver HiGHS

Said Guimarães Kenny Vinente

Programa de Pós-Graduação em Engenharia Elétrica,
Universidade Federal do Amazonas, Manaus, AM, Brasil

8 de julho de 2026

Abstract

Este relatório apresenta a análise da resolução de três problemas clássicos de escalonamento — *Machine Scheduling* (máquina única), *Job Shop* e *Flow Shop* — todos modelados e resolvidos em **Julia** com **JuMP** e o solver de código aberto **HiGHS**, com configuração livre do solver conforme permitido pelo enunciado. Em cada problema comparamos um método *exato* (programação inteira mista, MILO) com *heurísticas* construtivas, medindo tempo de execução, *status* de término e a distância ao ótimo conhecido. No Job Shop, trabalhamos com a formulação (*big-M*) que nos ajudou a fechar uma instância travada (*gap* de 16,2 % para 0 %) sem alterar solver nem tempo. No Flow Shop, formulado *sem big-M*, o *gap* ainda persistiu na maior instância, evidenciando que a raiz do problema é a natureza NP-difícil, e não apenas o *big-M*; na instância 10×20 , a heurística NEH (292) superou a melhor solução que o método exato conseguiu provar no tempo (297). Cada parte é relatada em três blocos — *o que foi realizado*, *os resultados na íntegra* e *o que foi percebido*.

Palavras-chave: escalonamento, otimização combinatória, Julia, JuMP, HiGHS, *makespan*, *gap* de otimalidade.

1 Metodologia geral

Os experimentos foram conduzidos em ambiente Windows, sobre Julia 1.10 com *kernel* Jupyter, utilizando o JuMP como camada de modelagem e o HiGHS como único solver, conforme exigido pelo enunciado. As instâncias de cada problema foram lidas das pastas `machinescheduling_instances` (Parte 1), `jsplib_subset` (Parte 2) e `fssp_problems` (Parte 3). A configuração do solver (limite de tempo, *gap*, tolerâncias) foi ajustada por problema, dentro da liberdade permitida.

Na Parte 1 usou-se o *gap* de otimalidade reportado pelo solver, nas partes 2 e 3, além desse *gap* de prova, mediu-se também a *distância ao ótimo conhecido*, isto é, o quanto o *makespan* obtido excede o valor de referência de cada instância — distinção essencial, pois uma instância pode *achar* o ótimo (distância nula) e ainda assim não *prová-lo* (*gap* alto).

2 Parte 1 — Machine Scheduling ($1 \mid r_j \mid \sum T_j$)

O que foi realizado.

A construção foi feita de baixo para cima: primeiro um *simulador* de cronograma; depois três *heurísticas* como linha de base — FIFO, EDD (menor prazo primeiro) e SPT (menor duração primeiro); e por fim o *modelo exato* MILO. O experimento central variou o **tamanho** da instância (n crescente) com **três seeds por tamanho** — para separar o efeito do tamanho do efeito da estrutura — medindo tempo, *status* do solver e a distância das heurísticas ao ótimo. O modelo foi validado na instância de referência do livro (`inst_book`, $n = 7$), de ótimo conhecido 16.

Resultados.

A configuração experimental está na Tabela 1 e a rodada completa, instância a instância, na Tabela 2. Nas linhas com *status* TIME_LIMIT, a coluna “ótimo/melhor” traz a melhor solução encontrada até o limite de tempo, e não um ótimo provado. O modelo foi validado na *inst_book* (ótimo conhecido 16, reproduzido). Duas leituras já saltam da tabela: o primeiro TIME_LIMIT aparece já em $n = 11$ e em apenas uma das três *seeds*, e nas instâncias grandes o método exato ainda encontra soluções bem melhores que as heurísticas (na *inst_n24_s01*, 145 contra 241 da melhor heurística).

Table 1: Parte 1 — configuração experimental (reprodutibilidade).

Item	Valor
Linguagem / ferramentas	Julia, JuMP, solver HiGHS
Limite de tempo	60 s por instância
Tamanhos (n)	{5, 7, 9, 11, 13, 15, 17, 19, 21, 24}
<i>Seeds</i> por tamanho	3 (mais a instância de referência <i>inst_book</i>)
Métricas coletadas	atraso FIFO/EDD/SPT; melhor valor exato; tempo; <i>status</i>
Validação	<i>inst_book</i> , ótimo conhecido = 16, reproduzido

Table 2: Parte 1 — rodada completa: atraso total das heurísticas (FIFO/EDD/SPT) vs. método exato, por instância. Em TIME_LIMIT, “ótimo/melhor” é a melhor solução encontrada, não provada. As linhas de $n = 13$ e $n = 15$ (:) foram omitidas na exibição do *DataFrame*.

Instância	n	FIFO	EDD	SPT	ótimo/melhor	tempo (s)	Status
<i>inst_n05_s01</i>	5	9	7	54	7	0,02	OPTIMAL
<i>inst_n05_s02</i>	5	19	15	15	15	0,15	OPTIMAL
<i>inst_n05_s03</i>	5	4	3	20	3	0,05	OPTIMAL
<i>inst_book</i>	7	31	27	51	16	0,31	OPTIMAL
<i>inst_n07_s01</i>	7	42	58	99	38	0,39	OPTIMAL
<i>inst_n07_s02</i>	7	40	31	77	30	0,76	OPTIMAL
<i>inst_n07_s03</i>	7	36	30	56	26	0,26	OPTIMAL
<i>inst_n09_s01</i>	9	38	29	62	26	0,65	OPTIMAL
<i>inst_n09_s02</i>	9	83	94	64	56	1,73	OPTIMAL
<i>inst_n09_s03</i>	9	47	39	43	32	5,65	OPTIMAL
<i>inst_n11_s01</i>	11	46	43	134	16	1,25	OPTIMAL
<i>inst_n11_s02</i>	11	98	113	229	62	11,67	OPTIMAL
<i>inst_n11_s03</i>	11	178	223	180	116	60,01	TIME_LIMIT
: ($n = 13$ e $n = 15$ — 6 linhas omitidas na exibição)							
<i>inst_n17_s01</i>	17	379	275	696	172	60,04	TIME_LIMIT
<i>inst_n17_s02</i>	17	232	273	599	162	60,03	TIME_LIMIT
<i>inst_n17_s03</i>	17	189	225	658	120	60,03	TIME_LIMIT
<i>inst_n19_s01</i>	19	396	434	1013	317	60,02	TIME_LIMIT
<i>inst_n19_s02</i>	19	207	290	491	128	60,02	TIME_LIMIT
<i>inst_n19_s03</i>	19	344	313	656	252	60,02	TIME_LIMIT
<i>inst_n21_s01</i>	21	301	235	746	200	60,04	TIME_LIMIT
<i>inst_n21_s02</i>	21	336	367	743	231	60,05	TIME_LIMIT
<i>inst_n21_s03</i>	21	297	435	801	220	60,02	TIME_LIMIT
<i>inst_n24_s01</i>	24	255	241	776	145	60,06	TIME_LIMIT
<i>inst_n24_s02</i>	24	380	377	903	287	60,05	TIME_LIMIT
<i>inst_n24_s03</i>	24	494	415	1053	294	60,04	TIME_LIMIT

O que foi percebido.

1. **Parede de complexidade real.** O tempo do exato cresce abruptamente com n (comportamento NP-

difícil): até $n = 9$ todas as instâncias fecham com prova (a mais lenta, `inst_n09_s03`, em 5,65 s); a partir de $n = 17$ nenhuma fecha em 60 s. Entre os dois regimes há uma faixa de transição.

2. **A transição depende da estrutura, não só do tamanho.** Em $n = 11$, mesmas dimensões, duas *seeds* fecham (1,25 s e 11,67 s) e a terceira (`inst_n11_s03`) estoura o tempo — a fronteira entre “fácil” e “intratável” não é um degrau limpo em um valor de n , e foi o que motivou as três *seeds* por tamanho.
3. **Nenhuma heurística campeã.** FIFO, EDD e SPT trocam de líder conforme a instância: o SPT vence na `inst_n09_s02` (64 contra 83 e 94), o FIFO vence na `inst_n11_s03` (178 contra 223 e 180); nenhuma domina, embora EDD e FIFO sejam em geral preferíveis ao SPT.
4. **Heurísticas ficam longe do ótimo.** Quando há comparação possível, a folga é grande (68 % no `inst_book`) — a busca exata compensa nas instâncias pequenas.

3 Parte 2 — Job Shop

O que foi realizado.

O modelo é disjuntivo com *big-M* e duas famílias de restrição — precedência dentro da tarefa e disjunção por máquina — validado na `ft06` (*makespan* 55, igual ao ótimo). Foram executadas 10 instâncias da biblioteca JSPLIB (famílias `ft/la/abz/orb`) com limite de 240 s. O tamanho do modelo (número de binárias) cresce com o formato (Tabela 3), o que já antecipa um achado: a `ft20`, apesar de “só” 20×5 , tem mais binárias que qualquer 10×10 . Conduziram-se três experimentos, mexendo em uma coisa de cada vez: (i) o método base com *big-M* frouxo; (ii) o efeito do tempo (60/120/240 s); e (iii) o efeito da formulação (apertar o *big-M*) e dos “botões” do HiGHS.

Table 3: Parte 2 — tamanho do modelo: número de variáveis binárias por formato de instância.

Formato	Binárias	Instâncias
6×6	90	<code>ft06</code>
10×5	225	<code>la01–la04</code>
10×10	450	<code>ft10</code> , <code>abz5</code> , <code>abz6</code> , <code>orb01</code>
20×5	950	<code>ft20</code>

Resultados.

O método base (Experimento i) resolveu cinco instâncias na otimalidade e travou cinco no limite de tempo (Tabela 4). O efeito do tempo (Experimento ii) está na Tabela 5: mais tempo melhora a solução, mas quase não fecha novas provas. O efeito da formulação (Experimento iii) está na Tabela 6, comparando o *big-M* frouxo com o tratamento final (*big-M* apertado + botões): o destaque é a `la03`, que fecha de 16,2 % para 0 % sem mudar solver nem tempo.

Table 4: Parte 2 — Experimento (i): método base, *big-M* frouxo, 240 s. “dist.” é a distância ao ótimo conhecido; *gap* é o *gap* de prova do solver.

Instância	Tam.	Ótimo	<i>Makespan</i>	dist.	<i>gap</i>	Status
ft06	6 × 6	55	55	0,0 %	0,0 %	OPTIMAL
la01	10 × 5	666	666	0,0 %	0,0 %	OPTIMAL (159 s)
la02	10 × 5	655	655	0,0 %	0,0 %	OPTIMAL (155 s)
la04	10 × 5	590	590	0,0 %	0,0 %	OPTIMAL (88 s)
abz6	10 × 10	943	943	0,0 %	0,0 %	OPTIMAL (88 s)
la03	10 × 5	597	597	0,0 %	16,2 %	TIME_LIMIT
ft20	20 × 5	1165	1265	8,6 %	62,1 %	TIME_LIMIT
ft10	10 × 10	930	970	4,3 %	23,5 %	TIME_LIMIT
abz5	10 × 10	1234	1239	0,4 %	16,6 %	TIME_LIMIT
orb01	10 × 10	1059	1113	5,1 %	30,0 %	TIME_LIMIT

Table 5: Parte 2 — Experimento (ii): efeito do tempo (distância ao ótimo conhecido; *big-M* frouxo).

Instância	60 s	120 s	240 s
la03	+4,2 %	+2,8 %	+0,0 % (<i>gap</i> 16,2 %)
ft20	+12,6 %	+8,6 %	+8,6 %
ft10	+8,3 %	+6,7 %	+4,3 %
abz5	+1,1 %	+0,4 %	+0,4 %
orb01	+13,8 %	+8,7 %	+5,1 %
la01	TIME_LIMIT	TIME_LIMIT	OPTIMAL (159 s)
la02	TIME_LIMIT	TIME_LIMIT	OPTIMAL (155 s)

Table 6: Parte 2 — Experimento (iii): *big-M* frouxo vs. tratamento final (*big-M* apertado + botões do HiGHS), 240 s.

Instância	Tam.	Ótimo	<i>big-M</i> frouxo		Tratamento final		Status final
			<i>mk</i>	<i>gap</i>	<i>mk</i>	<i>gap</i>	
ft06	6 × 6	55	55	0,0 %	55	0,0 %	OPTIMAL
la01	10 × 5	666	666	0,0 %	666	0,0 %	OPTIMAL
la02	10 × 5	655	655	0,0 %	655	0,0 %	OPTIMAL
la03	10 × 5	597	597	16,2 %	597	0,0 %	OPTIMAL
la04	10 × 5	590	590	0,0 %	590	0,0 %	OPTIMAL
abz6	10 × 10	943	943	0,0 %	943	0,0 %	OPTIMAL
ft20	20 × 5	1165	1265	62,1 %	1287	61,3 %	TIME_LIMIT
ft10	10 × 10	930	970	23,5 %	959	22,0 %	TIME_LIMIT
abz5	10 × 10	1234	1239	16,6 %	1239	18,4 %	TIME_LIMIT
orb01	10 × 10	1059	1113	30,0 %	1109	29,1 %	TIME_LIMIT

Observação honesta: o tratamento final não foi uniformemente melhor nas grandes. Na la03 foi vitória limpa (fechou); na ft20 chegou a piorar a *makespan* (1265 → 1287); nas demais, variou pouco. O ganho concentra-se onde o aperto do *big-M* efetivamente “morde”.

O que foi percebido.

1. **Solução ≠ prova; *gap* grande é prova fraca.** la01, la02 e la03 acham o ótimo cedo, mas o solver não o *prova*; a raiz é a relaxação frouxa do *big-M* (quanto maior o *M*, mais frouxa a prova).

2. **Mais tempo melhora a solução, mas quase não melhora a prova.** O teto da prova é imposto pela formulação, não pelo tempo.
3. **Apertar o M ataca a raiz — e o efeito depende da estrutura.** Fechou a $1a03$ ($16,2\% \rightarrow 0\%$, mesmo solver e tempo), mas quase não moveu as grandes.
4. **Estrutura importa mais que tamanho.** $abz5$ e $abz6$, ambas 10×10 : a $abz6$ fecha em 88 s, a $abz5$ não fecha; e a $ft20$ (950 binárias) tem o pior *gap* de todas.
5. **Os botões do solver rendem pouco.** Paralelismo, esforço de heurística e detecção de simetria deram ganho modesto e não-monótono; ligar os 8 núcleos não acelerou.
6. **Existe uma parede que mais tempo/ajuste não quebra.** As quatro grandes ($ft10$, $ft20$, $abz5$, $orb01$) não fecham em 240 s, o que é esperado — a $ft10$ é a 10×10 clássica que resistiu à comunidade por cerca de duas décadas.

Nota metodológica. Durante a coleta, um travamento por suspensão do computador fez a numeração de uma rodada pular a $ft10$, associando seu resultado ao rótulo da $ft20$ e gerando um *makespan* menor que o ótimo — impossível. O erro foi detectado exatamente pela checagem de sanidade (*makespan* nunca pode ser menor que o ótimo) e motivou a migração para captura automática dos dados.

4 Parte 3 — Flow Shop

O que foi realizado.

O Flow Shop é um caso particular do Job Shop em que *todas* as tarefas passam pelas máquinas na *mesma* ordem, reduzindo o problema a escolher uma única permutação. Aqui o modelo é **posicional e não usa big-M**: uma grade de binárias de atribuição (tarefa $\times$ posição) com os tempos encadeados por recursão. Voltam as heurísticas: NEH (insere tarefas por tempo total decrescente) e a regra de Johnson — esta última corretamente excluída por ser exata apenas para $m = 2$, enquanto todas as instâncias têm $m \geq 3$. Foram resolvidas 7 instâncias (formato $m \times n$) com limite de 120 s, comparando o exato com o NEH e validando contra um piso inferior calculado. Em seguida, aprofundou-se a instância que não fecha (a 10×20) num experimento de quatro rodadas.

Resultados.

A rodada completa está na Tabela 7: seis instâncias fecham com prova e apenas a 10×20 atinge o limite de tempo. Os valores reproduzem os de referência dos *slides* (126, 129, 263), confirmando a fidelidade da formulação. O experimento na 10×20 está na Tabela 8.

Table 7: Parte 3 — rodada completa nas 7 instâncias: modelo posicional (exato) vs. heurística NEH, 120 s. “piso” é o limite inferior calculado; “folga” = NEH – exato.

Inst. ($m \times n$)	piso	exato	NEH	folga	folga %	gap %	tempo (s)	Status
3×10	126	126	126	0	0,0	0,0	0,01	OPTIMAL
3×20	218	218	218	0	0,0	0,0	0,10	OPTIMAL
5×10	126	129	132	3	2,3	0,0	0,15	OPTIMAL
5×20	256	263	269	6	2,3	0,0	13,46	OPTIMAL
5×30	327	327	327	0	0,0	0,0	2,69	OPTIMAL
8×10	167	187	197	10	5,3	0,0	7,75	OPTIMAL
10×20	278	297	292	−5	−1,7	5,5	120,01	TIME_LIMIT

Table 8: Parte 3 — experimento na instância 10×20 (a única que não fecha). Ajudar o solver melhora a solução facilmente, mas o *gap* resiste e nunca fecha.

Rodada	Solução	<i>gap</i>	Tempo	Status
1. Base	297	5,5 %	120 s	TIME_LIMIT
2. <i>Warm start</i> (fila NEH)	292	3,6 %	120 s	TIME_LIMIT
3. Piso inferior	290	3,0 %	120 s	TIME_LIMIT
4. Tudo + 300 s + botões	292	3,2 %	300 s	TIME_LIMIT

O que foi percebido.

1. **A formulação é fiel.** Na 3×10 , solver, calculadora de *makespan* independente e piso calculado à mão chegaram todos a 126; os valores 126/129/263 batem com a referência.
2. **Sem *big-M* ≠ relaxação LP apertada.** Correção importante: a relaxação do modelo posicional também é frouxa (limite inicial de 19 na 3×10 , $\approx 85\%$ de *gap* no arranque); quem fecha o *gap* rapidamente é o *branch-and-bound* (1 nó, 0,02 s), não uma relaxação forte.
3. **Estrutura importa mais que tamanho.** A 5×30 (30 tarefas) fecha em 2,7 s, enquanto a 5×20 (menos tarefas) leva 13,5 s.
4. **Melhorar a solução é barato; fechar a prova é caro.** Na 10×20 , *warm start* e piso derrubaram a solução de 297 para 290 (abaixo do próprio NEH), mas o *gap* caiu apenas de 5,5 % para 3,0 % e nunca fechou, nem com 300 s.
5. **A causa do *gap* mudou, o sintoma permanece.** Não há *big-M* aqui; mesmo assim o *gap* resiste na instância grande — agora por causa da explosão combinatória das n^2 binárias da permutação. Mesmo sintoma (prova cara), causa diferente.
6. **Na maior instância, a heurística vence o exato.** O NEH (292) supera a melhor solução que o exato consegue provar no tempo (297 na base). O NEH acerta o ótimo em 3 das 7 instâncias e fica a $\leq 5,3\%$ nas demais.
7. **A regra de Johnson não se aplica.** Por ser exata só para $m = 2$ e como todas as instâncias têm $m \geq 3$, foi corretamente excluída — limitação do conjunto, não do método.